

Technical Stack Document

Citation-Grounded RAG Chatbot for Public Banking Documentation

A Production-Grade Retrieval-Augmented Generation System

February 07, 2026

Backend Architecture and Applied AI Systems Engineering

1. Abstract

This document presents the complete technical stack for a citation-grounded Retrieval-Augmented Generation (RAG) chatbot designed to answer procedural and policy-based questions on public banking documentation. The system addresses domains including Know Your Customer (KYC), re-KYC procedures, grievance escalation mechanisms, chargeback processes, dispute resolution, loans, credit cards, forms, and timelines.

The architecture implements a production-grade RAG pipeline using Python-based microservices with FastAPI for API orchestration, sentence-transformers for embedding generation, Qdrant as the vector database, and Falcon-7B-Instruct served via vLLM for answer generation. The system enforces strict citation grounding, ensuring every response includes traceable references to source documents with page and section identifiers.

Key technical characteristics include: asynchronous request handling with streaming token generation; structure-aware document chunking preserving procedural semantics; metadata-filtered semantic retrieval; optional cross-encoder reranking; scheduled incremental ingestion with hash-based change detection; and comprehensive logging infrastructure for retrieval traces and generation audits.

The stack is engineered for production deployment with emphasis on answer accuracy, citation correctness, response latency optimization, and compliance with data acquisition policies. This document provides implementation-level detail on each layer, engineering rationale for technology choices, and evaluation methodologies for system validation.

2. System Overview

The Citation-Grounded RAG Chatbot operates as a question-answering system over public banking documentation. Unlike general-purpose conversational agents, this system enforces strict grounding to retrieved documents and mandates citation generation for every factual claim.

2.1 Core Functional Requirements

The system must:

- Accept natural language queries regarding banking procedures and policies
- Retrieve relevant document segments from a versioned corpus
- Generate accurate, citation-grounded answers using a locally-served LLM
- Include document name, page number, and section identifiers in all citations
- Stream responses token-by-token for reduced perceived latency
- Periodically acquire and index updated documentation
- Maintain audit logs of retrieval and generation operations

2.2 Data Scope and Sources

The documentation corpus consists exclusively of publicly available, customer-facing materials from banking institutions. This includes:

- KYC and re-KYC procedure manuals
- Grievance escalation policy documents
- Chargeback and dispute resolution guidelines
- Loan application and processing documentation
- Credit card terms, conditions, and procedural guides
- Regulatory forms and submission timelines

All documents are acquired through compliant web scraping with robots.txt adherence, domain restriction, and keyword-based filtering. No proprietary or non-public data is ingested.

2.3 Architectural Principles

The system architecture adheres to the following design principles:

Citation Enforcement: The LLM prompt structure and post-processing pipeline enforce citation generation. Answers without valid citations are rejected or flagged.

Semantic Retrieval with Metadata Filtering: Vector similarity search is augmented with metadata-based filtering (document category, version, date) to improve relevance and support temporal queries.

Structure-Aware Chunking: Document segmentation preserves procedural semantics by respecting heading hierarchies, numbered steps, and bulleted lists rather than applying naive fixed-size windowing.

Asynchronous and Streaming Execution: All I/O-bound operations (retrieval, LLM inference) use async patterns. Token streaming reduces time-to-first-token and improves user experience.

Incremental Update Model: The ingestion pipeline computes content hashes and selectively re-indexes only modified documents, minimizing computational overhead and preserving vector database consistency.

3. Architectural Design

3.1 High-Level Architecture

The system follows a layered architecture with clear separation of concerns:

Layer 1: API and Orchestration Layer

- FastAPI server handling HTTP requests
- Async request routing and streaming response management
- RAG pipeline orchestration

Layer 2: Data Acquisition and Ingestion Layer

- Web scraping modules with compliance checks
- Document download and hash computation
- Cloud storage upload and versioning

Layer 3: Processing and Indexing Layer

- PDF parsing and text extraction
- Structure-aware chunking with metadata tagging
- Embedding generation and vector database insertion

Layer 4: Retrieval Layer

- Query embedding generation
- Metadata-filtered vector search in Qdrant
- Optional cross-encoder reranking

Layer 5: Generation Layer

- Context assembly and prompt construction
- LLM inference via vLLM server
- Citation extraction and validation

Layer 6: Observability Layer

- Structured logging of all operations
- Retrieval trace storage

- Latency measurement and aggregation

3.2 Data Flow

Ingestion Flow:

1. Scheduled scraper identifies new or modified documents
2. Documents are downloaded and stored in cloud object storage with version metadata
3. PDF processing pipeline extracts page-wise text
4. Structure-aware chunker segments text into semantically coherent blocks
5. Chunks are embedded using sentence-transformers model
6. Vectors and metadata are inserted into Qdrant with document version tags

Query Flow:

1. User query arrives via FastAPI endpoint
2. Query is embedded using the same sentence-transformers model
3. Qdrant performs semantic search with optional metadata filters
4. Top-K chunks are retrieved and optionally reranked
5. Retrieved chunks are assembled into context with citation metadata
6. Context and query are formatted into LLM prompt
7. vLLM server generates response with streaming enabled
8. Tokens are streamed to client via FastAPI StreamingResponse
9. Citations are extracted and validated in post-processing
10. Retrieval and generation logs are persisted

4. Backend and API Layer

4.1 Technology: FastAPI

FastAPI is selected as the web framework for its native async support, automatic OpenAPI documentation generation, Pydantic-based request/response validation, and high performance on I/O-bound workloads. The framework's async capabilities are critical for handling concurrent requests during LLM inference, which involves waiting on external vLLM server responses.

4.2 API Design

The API exposes the following endpoints:

POST /query - Main query endpoint

- Accepts JSON payload with query text and optional metadata filters
- Returns streaming response with Server-Sent Events (SSE) or chunked transfer encoding
- Each chunk contains a token or citation metadata

POST /query-sync - Synchronous query endpoint

- Returns complete response with citations as JSON
- Used for batch evaluation or non-streaming clients

GET /health - Health check endpoint

- Verifies connectivity to Qdrant and vLLM server
- Returns status and latency measurements

POST /admin/ingest - Manual ingestion trigger

- Triggers immediate scraping and indexing
- Protected by API key authentication

4.3 Request Handling and Async Patterns

FastAPI's async request handlers enable non-blocking I/O. This pattern ensures that the server can handle multiple concurrent queries without blocking on LLM inference.

4.4 Error Handling and Validation

Pydantic models enforce input validation:

- Query length constraints (min 5, max 500 characters)

- Metadata filter type validation
- Response schema enforcement with citation fields

Error responses include structured error codes:

- RETRIEVAL_FAILED: Qdrant connection or query failure
- LLM_TIMEOUT: vLLM server timeout
- NO_CITATIONS: Generated answer lacks valid citations
- INVALID_QUERY: Malformed or out-of-scope query

5. Data Acquisition Layer

5.1 Web Scraping Strategy

The acquisition layer uses requests for HTTP operations and BeautifulSoup for HTML parsing. The scraping strategy implements:

Robots.txt Compliance: Before crawling any domain, the scraper fetches and parses robots.txt to respect disallow directives and crawl delays.

Keyword-Based Filtering: URLs are crawled only if they match banking-related keywords ("KYC", "chargeback", "grievance", "loan", "credit-card", etc.). This reduces noise and focuses on relevant documentation.

Domain Restriction: The scraper maintains a whitelist of official banking institution domains. Cross-domain links are ignored.

Public Document Detection: Only documents accessible without authentication are scraped. The scraper identifies login walls and skips protected resources.

5.2 Implementation

The scraper module implements the following workflow:

1. Fetch seed URLs from configuration
2. Parse robots.txt and extract crawl rules
3. Crawl seed pages and extract document links
4. Filter links by keyword and domain
5. Download PDFs with retry logic and exponential backoff
6. Compute SHA-256 hash of each document
7. Compare hash against previously indexed versions
8. Upload new or modified documents to cloud storage

5.3 Rate Limiting and Politeness

The scraper implements rate limiting to avoid overloading source servers:

- Minimum 2-second delay between requests to the same domain
- Maximum 5 concurrent connections per domain
- Exponential backoff on HTTP 429 (Too Many Requests) responses
- User-Agent header identifying the bot with contact information

5.4 Error Handling

The scraper logs all HTTP errors, DNS failures, and timeout events. Failed downloads are retried up to 3 times with exponential backoff. Persistent failures are logged to a dead-letter queue for manual review.

6. Document Storage and Versioning

6.1 Technology: Cloud Object Storage

Documents are stored in Amazon S3 or Google Cloud Storage. Object storage is chosen over a relational database or file system for:

- Unlimited scalability for document corpus growth
- Built-in versioning and lifecycle management
- High availability and durability (11 nines durability for S3)
- Native integration with processing workflows (Lambda/Cloud Functions)

6.2 Object Naming and Organization

Objects are organized with the following key structure:

`documents/<domain>/<document_hash>/metadata.json`

`documents/<domain>/<document_hash>/document.pdf`

The hash-based naming ensures content-addressable storage. Metadata includes:

- `source_url`: Original download URL
- `acquisition_timestamp`: UTC timestamp of download
- `document_hash`: SHA-256 hash of PDF content
- `document_category`: Manually tagged or inferred category
- `version_id`: Object version identifier from cloud storage

6.3 Versioning and Change Detection

Cloud storage versioning is enabled, allowing rollback to previous document versions. The ingestion pipeline performs hash-based change detection:

1. Compute SHA-256 hash of newly downloaded document
2. Query metadata store for existing documents from same URL
3. Compare hashes
4. If hash differs, upload new version and trigger re-indexing
5. If hash matches, skip processing

This ensures that only modified documents are re-processed, minimizing computational cost and preventing vector database pollution with duplicate embeddings.

6.4 Retention and Lifecycle Policies

Object lifecycle policies are configured to:

- Transition documents older than 90 days to infrequent access storage
- Retain all versions for 365 days for audit purposes
- Archive versions older than 365 days to cold storage

7. Document Processing and Chunking

7.1 PDF Parsing: PyMuPDF vs pdfplumber

Two libraries are evaluated for PDF text extraction:

PyMuPDF (fitz): C-based library with high performance and accurate text extraction. Preserves text layout and supports page-wise extraction. Chosen as the primary parser for speed and accuracy.

pdfplumber: Python-native library with table extraction capabilities. Used as a fallback when PyMuPDF fails or for documents with complex table structures.

7.2 Page-Wise Text Extraction

Text is extracted page-by-page to preserve page number metadata for citations. Each page's text is stored with:

- `page_number`: 1-indexed page number
- `text`: Extracted text content
- `document_id`: Reference to parent document

7.3 Structure-Aware Chunking Strategy

Naive fixed-size chunking (e.g., 512-token windows) breaks procedural semantics. The system implements a custom structure-aware chunker that:

Heading-Based Segmentation: Identifies section headings using heuristics:

- Font size and weight changes (for parsed PDF layout)
- Regex patterns for numbered sections (e.g., '1.1 KYC Requirements')
- All-caps lines shorter than 100 characters

Numbered Procedure Grouping: Detects numbered lists ("1.", "2.", etc.) and groups consecutive steps into a single chunk. This preserves procedural sequences.

Bulleted List Grouping: Identifies bullet points ("-", "•", "*") and groups related items.

Procedural Block Extraction: Recognizes common procedural patterns and keeps such blocks intact as a single chunk.

7.4 Chunk Metadata

Each chunk is tagged with:

- `document_name`: Human-readable document title
- `page_number`: Page(s) where chunk appears

- `section_title`: Inferred section heading
- `chunk_type`: 'heading', 'procedure', 'list', 'paragraph'
- `category`: 'KYC', 'chargeback', 'loan', etc.
- `document_hash`: Content hash for version tracking

7.5 Chunking Pipeline

1. Extract text page-by-page from PDF
2. Apply heading detection to identify section boundaries
3. Detect numbered and bulleted lists
4. Group consecutive list items
5. Detect procedural blocks with step sequences
6. Split long paragraphs at sentence boundaries (max 512 tokens per chunk)
7. Tag each chunk with metadata
8. Validate that all chunks have page numbers

8. Embedding and Retrieval Layer

8.1 Embedding Model: sentence-transformers

The system uses models from the sentence-transformers library, specifically BERT-family models such as bge-small-en-v1.5 or all-MiniLM-L6-v2.

Model Selection Rationale:

- **bge-small-en-v1.5:** 33M parameters, 384-dimensional embeddings, strong performance on semantic search benchmarks
- **all-MiniLM-L6-v2:** 22M parameters, 384-dimensional embeddings, faster inference, suitable for real-time query embedding

BERT-family models are chosen over larger models for:

- Low latency query encoding (<50ms on CPU)
- Modest GPU memory requirements during indexing
- Strong performance on domain-specific procedural text

8.2 Embedding Generation

Embeddings are generated for:

Document Chunks (Offline): During indexing, each chunk is embedded using the sentence-transformers model. Embeddings are computed in batches of 32 to leverage GPU parallelism.

User Queries (Online): At query time, the user's question is embedded using the same model. The embedding is computed on CPU or GPU depending on deployment configuration.

To ensure embedding consistency, the same model checkpoint is used for both indexing and retrieval.

8.3 Vector Database: Qdrant

Qdrant is selected as the vector database for:

- Native support for metadata filtering (critical for filtering by document category or version)
- High-performance HNSW (Hierarchical Navigable Small World) indexing
- Payload storage alongside vectors (stores chunk text and metadata)
- REST and gRPC APIs with async client support
- On-disk persistence with configurable memory mapping

Alternative vector databases (Weaviate, Milvus, Pinecone) were considered. Qdrant is chosen for its simplicity, strong metadata filtering, and absence of vendor lock-in.

8.4 Collection Schema

A Qdrant collection is created with the following schema:

- **Vector:** 384-dimensional float32 vector
- **Payload:** JSON object containing chunk text, document name, page number, section title, category, document hash, and chunk ID

8.5 Indexing Configuration

Qdrant is configured with:

- **HNSW parameters:** m=16, ef_construct=100
- **Distance metric:** Cosine similarity
- **On-disk storage:** Enabled for large collections (>1M vectors)

8.6 Retrieval Strategy

At query time, the system performs:

Step 1: Query Embedding - Convert user query to vector using sentence-transformers

Step 2: Metadata Filtering (Optional) - Apply filters for document category or version

Step 3: Top-K Similarity Search - Qdrant performs HNSW search with configurable accuracy-speed tradeoff

Step 4: Result Extraction - Extract chunk text, document name, page number, and similarity score

8.7 Top-K Selection

The system retrieves K=10 chunks by default. This balances context richness (more chunks provide more information), LLM context window limits (Falcon-7B has 2048 token limit), and latency (more chunks increase LLM inference time).

9. Reranking Strategy

9.1 Why Reranking

Semantic search using bi-encoders (sentence-transformers) retrieves candidates based on vector similarity. However, bi-encoders may rank syntactically similar but semantically irrelevant chunks highly. Reranking with a cross-encoder improves precision.

9.2 Cross-Encoder Reranking

A cross-encoder model (e.g., cross-encoder/ms-marco-MiniLM-L-6-v2) is optionally applied to rerank the top-K retrieved chunks.

Cross-Encoder Mechanism:

- Takes query-chunk pairs as input
- Outputs a relevance score for each pair
- Uses full self-attention across query and chunk tokens
- More accurate than bi-encoder cosine similarity but slower

9.3 Reranking Pipeline

1. Retrieve top-K=20 chunks from Qdrant using bi-encoder
2. For each chunk, compute relevance score using cross-encoder
3. Sort chunks by cross-encoder score
4. Select top-10 reranked chunks
5. Pass to LLM generation

9.4 Computational Trade-off

Cross-encoder reranking adds latency (10-50ms per chunk on GPU). The system makes this optional via configuration:

- **Enabled:** Higher precision, suitable for complex queries
- **Disabled:** Lower latency, suitable for simple queries

10. Generation and LLM Serving Layer

10.1 Model: Falcon-7B-Instruct

Falcon-7B-Instruct is selected as the generation model for:

- Open-source license (Apache 2.0)
- Strong instruction-following capabilities
- 7B parameter size enables deployment on single-GPU instances
- Performance competitive with commercial models on factual QA tasks

10.2 Serving Infrastructure: vLLM

vLLM is used to serve Falcon-7B-Instruct. vLLM provides:

- **PagedAttention:** Memory-efficient KV cache management, enabling higher throughput
- **Continuous batching:** Dynamic batching of requests for GPU utilization
- **OpenAI-compatible API:** Standardized interface for LLM calls
- **Streaming support:** Token-by-token streaming for reduced latency

10.3 vLLM Server Configuration

The vLLM server is deployed with:

- `tensor-parallel-size 1`: Single-GPU deployment
- `dtype float16`: Half-precision for 2x memory reduction
- `max-model-len 2048`: Context window limit for Falcon-7B
- `gpu-memory-utilization 0.9`: Aggressive memory usage for maximum batch size

10.4 Prompt Engineering for Citation Grounding

The prompt structure enforces citation generation. The system instructs the LLM to answer using ONLY the provided context and MUST cite every factual claim using the format [Document Name, Page X]. Retrieved chunks are formatted with document name, page number, and section identifiers to provide the LLM with both content and citation metadata.

10.5 Generation Parameters

The LLM is called with:

- `temperature=0.1`: Low temperature for deterministic, factual answers
- `top_p=0.9`: Nucleus sampling for slight diversity

- max_tokens=512: Sufficient for procedural answers with citations
- stream=True: Enable token streaming

10.6 Streaming Token Generation

vLLM streams tokens incrementally. This reduces time-to-first-token from 2-3 seconds (full generation) to <500ms.

10.7 Citation Extraction and Validation

After generation, the system:

1. Parses the LLM output for citation patterns: [Document Name, Page X]
2. Validates that cited page numbers exist in the retrieved context
3. Flags responses without citations as CITATION_MISSING
4. Returns structured response with answer text and citation list

11. RAG Orchestration Pipeline

11.1 Custom Python RAG Pipeline

The RAG pipeline is implemented as a Python class that orchestrates all components: embedding model, Qdrant client, vLLM client, and optional reranker. The pipeline handles query embedding, chunk retrieval, optional reranking, context assembly, and answer generation in a unified workflow.

11.2 Intent Routing

The pipeline optionally includes intent classification:

- **Procedural queries:** 'How do I submit a chargeback?' → Full RAG pipeline
- **Definition queries:** 'What is re-KYC?' → Direct retrieval without extensive context
- **Out-of-scope queries:** 'What is the weather?' → Rejection with error message

11.3 Context Assembly

The context assembler sorts chunks by relevance score, formats each chunk with citation metadata, truncates context to fit LLM token limit (2048 tokens), and ensures at least 3 chunks are included for answer diversity.

11.4 Error Handling and Fallbacks

The pipeline includes error handling for no retrieval results, LLM timeouts, and malformed LLM output.

12. Streaming and Real-Time Serving

12.1 FastAPI StreamingResponse

FastAPI's `StreamingResponse` enables token-by-token streaming. The response uses Server-Sent Events (SSE) format, allowing the client (browser or API consumer) to receive tokens incrementally and render them in real-time.

12.2 Latency Optimization

Streaming reduces perceived latency:

- **Time-to-first-token:** <500ms (embedding + retrieval + first token generation)
- **Full response time:** 2-4 seconds (for 200-token answer)
- **Non-streaming baseline:** 3-5 seconds (must wait for complete generation)

12.3 Connection Management

The server handles client disconnections gracefully by detecting closed connections during streaming, canceling ongoing LLM generation, and releasing GPU memory and vLLM batch slots.

13. Scheduling and Incremental Updates

13.1 Scheduling Technology: APScheduler

APScheduler is used to schedule periodic scraping and indexing jobs. It provides cron-like scheduling with timezone support, in-process background job execution, and job persistence and retry logic.

13.2 Scheduled Jobs

The system configures the following jobs:

- **Daily Scraping Job:** Runs at 2 AM UTC
- **Weekly Full Re-Indexing Job:** Runs on Sundays at 3 AM UTC

13.3 Incremental Ingestion Workflow

The incremental ingestion pipeline downloads documents, computes hashes, compares against metadata store, and processes only new or modified documents. For modified documents, old chunks are marked as deprecated and new chunks are inserted with updated document hash.

13.4 Selective Re-Indexing

Only modified documents are re-indexed by tagging each chunk with `document_hash`, querying Qdrant for chunks with old hash, deleting old chunks, and inserting new chunks. This avoids full re-indexing, which would be computationally expensive for large corpora.

13.5 Atomic Updates

To prevent serving stale data during updates, new chunks are inserted with a `status=staging` tag, old chunks are marked `status=deprecated`, and after all new chunks are inserted, the system atomically flips staging to active and deletes deprecated.

14. Logging, Monitoring and Auditing

14.1 Structured Logging

The system uses Python's logging module with JSON formatters for structured logs, enabling easy parsing and analysis by centralized logging systems.

14.2 Logged Events

The system logs:

- **Query Events:** Query text, retrieval results, reranking scores, generated answers, citations, and latency breakdown
- **Ingestion Events:** Document downloads, hash comparisons, chunking times, embedding times, and Qdrant insertion status
- **Error Events:** Retrieval failures, LLM timeouts, citation extraction failures, and scraping errors

14.3 Retrieval Trace Logs

For each query, a retrieval trace is stored containing query ID, query text, retrieved chunks with IDs and scores, reranked scores (if enabled), context length in tokens, LLM latency, and total latency.

14.4 Latency Measurement

The system measures query embedding time, Qdrant search time, reranking time, context assembly time, LLM inference time (time-to-first-token and total), and end-to-end latency.

14.5 Ingestion Audit Logs

Every ingestion run is logged with timestamp, number of documents scraped, number of new documents, number of modified documents, number of chunks indexed, and total ingestion time.

14.6 Monitoring Integration

Logs are exported to centralized logging systems (e.g., Elasticsearch, CloudWatch Logs) for real-time alerting on error rates, latency percentile tracking (p50, p95, p99), query volume monitoring, and retrieval quality metrics.

15. Evaluation Methodology

15.1 Evaluation Dimensions

The system is evaluated on:

1. Citation Correctness: Do generated citations match retrieved chunks?
2. Retrieval Relevance: Are retrieved chunks relevant to the query?
3. Answer Completeness: Does the answer address all parts of the query?
4. Latency: Time-to-first-token and total response time
5. Failure Handling: Behavior on out-of-scope or ambiguous queries

15.2 Citation Correctness Evaluation

A test set of 200 queries with ground-truth answers is curated. For each query, the system generates an answer, extracts citations, verifies that cited page numbers exist in the retrieved chunks, and verifies that cited content matches the answer claims. Metric: Citation accuracy = (Correct citations) / (Total citations).

15.3 Retrieval Relevance Evaluation

For each test query, the system retrieves top-10 chunks, manually labels each chunk as Relevant / Partially Relevant / Irrelevant, and computes Precision@K and NDCG@K.

15.4 Answer Completeness Evaluation

Human evaluators rate answers on a 1-5 scale from completely incorrect to complete, correct, and well-explained. Metric: Average completeness score.

15.5 Latency Measurement

For a sample of 100 queries, the system measures time-to-first-token (p50, p95, p99), total response time (p50, p95, p99), and compares streaming vs non-streaming latency.

15.6 Failure Behavior Evaluation

The system is tested on out-of-scope queries, ambiguous queries, and queries with no relevant documents. Evaluation assesses whether the system correctly identifies out-of-scope queries, requests clarification for ambiguous queries, and returns appropriate fallback responses for zero-result scenarios.

16. Scalability and Reliability Considerations

16.1 Horizontal Scaling

The architecture supports horizontal scaling:

- **API Layer:** Multiple FastAPI instances behind a load balancer (e.g., NGINX, ALB)
- **vLLM Server:** Multiple vLLM instances with request routing based on load
- **Qdrant:** Qdrant cluster with sharding for large collections (>10M vectors)

16.2 Fault Tolerance

- **Retry Logic:** All external calls include exponential backoff retry
- **Circuit Breaker:** If vLLM server fails repeatedly, the system returns cached fallback responses
- **Health Checks:** The /health endpoint monitors dependencies

16.3 Caching

- **Query Embedding Cache:** Frequently asked queries are cached with their embeddings
- **Retrieval Result Cache:** Top-K results for common queries are cached with TTL=1 hour
- **LLM Response Cache:** Identical queries return cached responses for 5 minutes

16.4 Rate Limiting

The API enforces rate limits:

- 10 requests/minute per IP address
- 100 requests/hour per API key

16.5 Database Scalability

Qdrant supports horizontal sharding for collections >10M vectors, read replicas for high query throughput, and on-disk storage for memory efficiency.

16.6 GPU Resource Management

vLLM's continuous batching ensures efficient GPU utilization. The system monitors GPU memory and adjusts batch sizes dynamically.

17. Security, Privacy and Compliance Considerations

17.1 Data Privacy

- **Public Data Only:** The system ingests only publicly available documentation
- **Query Anonymization:** User queries are logged without IP addresses or user identifiers
- **Data Retention:** Logs are retained for 90 days and then deleted

17.2 Access Control

- **API Key Authentication:** The admin ingestion endpoint requires API key authentication
- **HTTPS Only:** All API traffic is encrypted with TLS

17.3 Compliance

- **Robots.txt Adherence:** The scraper respects robots.txt to comply with website policies
- **Copyright Compliance:** Only publicly accessible documents are scraped; no paywall bypassing

17.4 Rate Limiting and Abuse Prevention

- **IP-Based Rate Limiting:** Prevents DDoS and abuse
- **Input Validation:** Query length limits and content filtering prevent injection attacks

17.5 Secure Deployment

- **Infrastructure:** All services deployed in private VPC with firewall rules
- **Secrets Management:** API keys and credentials stored in secret managers

18. Engineering Trade-offs and Design Decisions

18.1 Open-Source LLM vs Commercial API

Decision: Use Falcon-7B-Instruct served via vLLM

Pros: No API costs, full control, data privacy, no rate limits

Cons: GPU infrastructure costs, lower performance than GPT-4 or Claude

Rationale: For a public banking documentation chatbot, data privacy and cost predictability outweigh the performance gap.

18.2 Qdrant vs Pinecone/Weaviate

Decision: Use Qdrant

Pros: Self-hosted, strong metadata filtering, no vendor lock-in

Cons: Requires infrastructure management, no managed scaling

Rationale: Self-hosting aligns with privacy requirements and avoids usage-based pricing.

18.3 Structure-Aware Chunking vs Fixed-Size Chunking

Decision: Implement structure-aware chunking

Pros: Preserves procedural semantics, improves retrieval relevance

Cons: Increased complexity, harder to debug

Rationale: Procedural documentation loses meaning when split arbitrarily.

18.4 Streaming vs Synchronous Responses

Decision: Default to streaming

Pros: Reduced perceived latency, better user experience

Cons: More complex error handling, client compatibility issues

Rationale: Streaming improves perceived responsiveness, critical for interactive chatbots.

18.5 Cross-Encoder Reranking vs Bi-Encoder Only

Decision: Make reranking optional

Pros (reranking): Higher retrieval precision

Cons (reranking): Added latency (10-50ms)

Rationale: For simple queries, bi-encoder retrieval is sufficient. For complex queries, the precision gain justifies the latency cost.

18.6 Incremental vs Full Re-Indexing

Decision: Incremental ingestion with hash-based change detection

Pros: Faster updates, lower compute costs

Cons: More complex logic, potential for inconsistency

Rationale: Most documents change infrequently. Full re-indexing is wasteful.

19. Limitations of the Current Stack

19.1 LLM Limitations

- **Context Window:** Falcon-7B has a 2048-token limit, constraining the amount of retrieved context
- **Reasoning Capability:** Falcon-7B struggles with multi-hop reasoning queries
- **Citation Reliability:** The LLM occasionally generates hallucinated citations despite prompt engineering

19.2 Retrieval Limitations

- **Cold Start Problem:** New documents are not indexed until the next scheduled scraping job
- **Semantic Drift:** Embedding models may not capture domain-specific banking terminology effectively
- **Multi-Document Queries:** Queries requiring synthesis across multiple documents are challenging

19.3 Scalability Limitations

- **GPU Bottleneck:** vLLM throughput is limited by GPU availability
- **Qdrant Single-Node:** Current deployment is single-node, limiting throughput for very high query volumes

19.4 Evaluation Limitations

- **Manual Annotation:** Evaluation requires manual labeling of test queries, which is time-consuming
- **Coverage Gaps:** Test set may not cover all edge cases

20. Future Extensions of the Stack

20.1 Hybrid Search

Combine semantic search with BM25 keyword search for better recall on exact-match queries (e.g., "Form 17" or "Section 3.2"). Implementation: Use Qdrant's payload indexing for keyword search, then combine scores with semantic search.

20.2 Multi-Hop Reasoning

Upgrade to a larger LLM (e.g., Llama-3-70B, Mixtral-8x7B) with larger context windows (8K+ tokens) to support multi-hop queries. Implementation: Serve larger models with tensor parallelism across multiple GPUs.

20.3 Fine-Tuning Embedding Models

Fine-tune sentence-transformers on domain-specific banking documentation to improve retrieval relevance. Implementation: Curate positive and negative query-chunk pairs, fine-tune using contrastive loss.

20.4 Query Intent Classification

Implement a query intent classifier to route queries to specialized sub-pipelines (e.g., definition queries vs procedural queries). Implementation: Train a small BERT classifier on labeled query data.

20.5 Multi-Modal Support

Extend the system to process images and tables in PDF documents. Implementation: Use Vision-Language Models (e.g., LLaVA) to extract structured data from tables and diagrams.

20.6 Federated Retrieval

Support queries across multiple document collections (e.g., internal policy documents + external regulatory documents). Implementation: Create separate Qdrant collections with cross-collection search.

20.7 User Feedback Loop

Implement a feedback mechanism where users rate answers, and low-rated responses are logged for model improvement. Implementation: Store feedback in a database, periodically retrain embedding models or rerankers on feedback data.

20.8 Multilingual Support

Extend the system to support queries and documents in multiple languages. Implementation: Use multilingual embedding models (e.g., paraphrase-multilingual-MiniLM) and multilingual LLMs (e.g., aya-101).

20.9 Explainability Enhancements

Provide users with retrieval scores and confidence indicators for generated answers. Implementation: Surface retrieval scores in the UI, add confidence scores based on citation count and chunk relevance.

20.10 Real-Time Document Monitoring

Implement webhook-based ingestion for immediate indexing when documents are updated. Implementation: Expose a webhook endpoint that triggers ingestion on document upload events.